Lab #3

Objectives

- Review lecture notes of discrete-time signal and system
- Implement script-based manipulation functions in SciLab for discrete-time signals

Report

1. Your report must include all script's source codes.
2. For each function, you have to try with an example and capture the output screen as evidence.
3. Finally, you upload your report to BKeL on time. You should down-size the image file to reduce the report file in order to be able to submit it in BKeL.

PART 1. SCILAB SCRIPT

A script function in Scilab will be declared under the following format

function [rv1 rv2.... rvn] = **Function_Name** (pv1, pv2,..., pvn)

[function body]

endfunction

where

rv1, rv2: return values

pv1, pv2: input parameters

Further references:

- https://help.scilab.org/doc/5.5.2/en_US/function.html
- https://help.scilab.org/doc/5.5.2/en_US/deff.html

PART 2. EXERCISES

Students implement the following functions in SciLab for discrete-time signals

Exercise 1. **function** [yn, yorigin] = **delay** (xn, xorigin, k) performs delay operation y(n) = x(n - k), where k > 0,

- The discrete-time signal x(n) is presented by vector xn
- xorigin indicates the origin's position of signal x(n).
- yn is vector of the output signal
- yorigin indicates the origin's position of signal y(n).

- x(n) and y(n) are graphically displayed in the same figure.

For example: To determine the delay signal **y(n) = x(n – 1)** where **x(n)={1, -2, 3↑, 6}.**
You can use your own function by calling

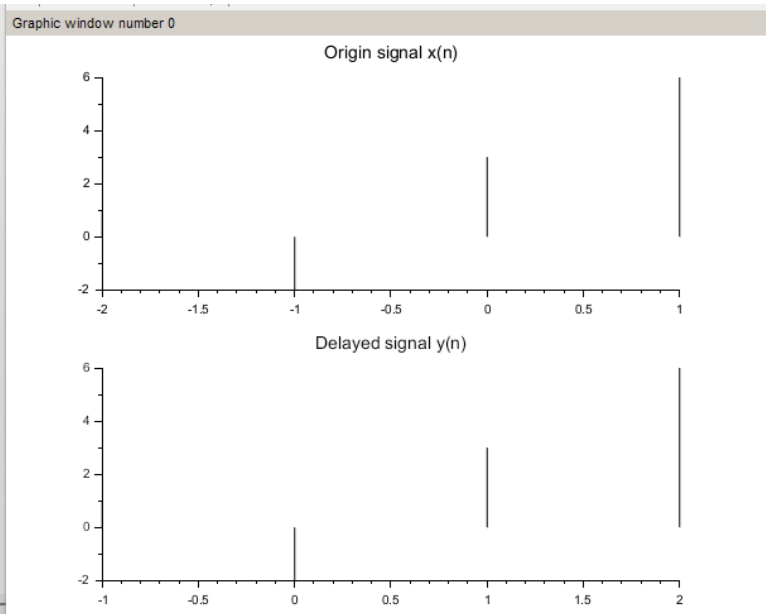
[yn, yorigin] = delay ([1, -2, 3, 6], 3, 1)**.**

Then, the output will be

- yn = [1, -2, 3, 6]
- yorigin = 2

Additionally, you have to graphically display x(n) and y(n) in the same figure.

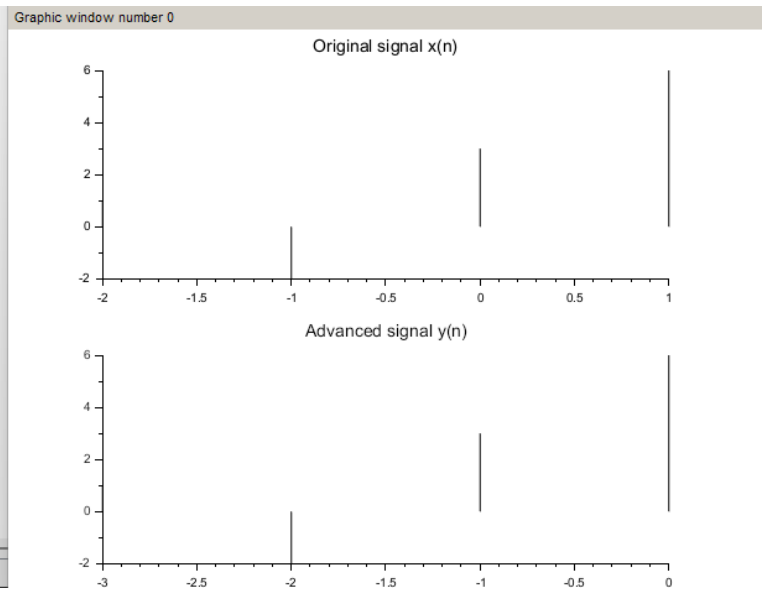
```
--> exec ('ex1.sci')

--> function [yn, yorigin] = delay(xn, xorigin, k)
-->     yn = xn;
-->     yorigin = xorigin - k;
-->     nx = (1:length(xn)) - xorigin;
-->     ny = (1:length(yn)) - yorigin;
-->
-->     subplot(2,1,1);
-->     plot2d3(nx, xn);
-->     title("Origin signal x(n)")
-->
-->     subplot(2,1,2);
-->     plot2d3(ny, yn);
-->     title("Delayed signal y(n)");
--> endfunction
Warning : redefining function: delay

--> delay ([1, -2, 3, 6], 3, 1)

 ans = [1x4 double]

   1.  -2.   3.   6.

-->
```



Exercise 2. **function** [yn, yorigin] = **advance** (xn, xorigin, k) performs advance operation y(n) = x(n + k), where k > 0 and

- The discrete-time signal x(n) is presented by vector xn
- xorigin indicates the origin's position of signal x(n).
- yn is vector of the output signal
- yorigin indicates the origin's position of signal y(n).
- x(n) and y(n) are graphically displayed in the same figure.

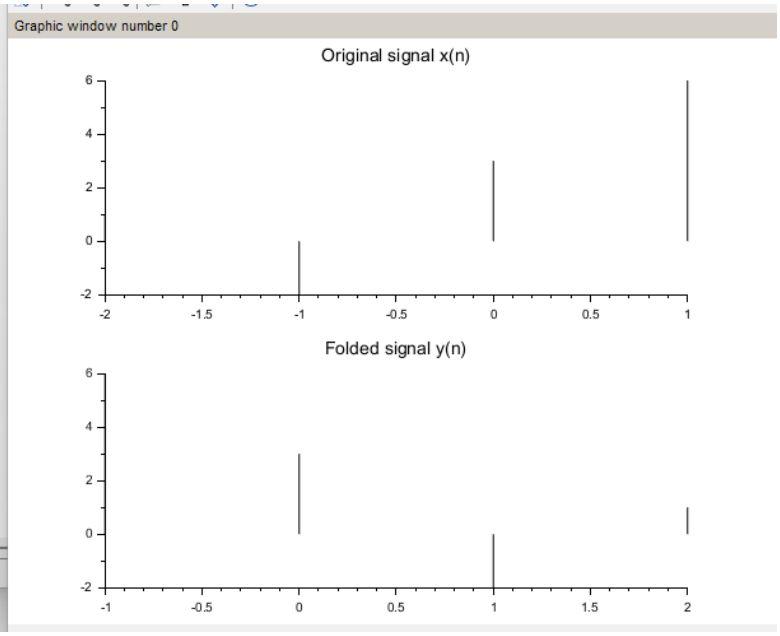
```
--> exec ('ex2.sci')

--> function [yn, yorigin] = advance(xn, xorigin, k)
-->     yn = xn;
-->     yorigin = xorigin + k;
-->     nx = (1:length(xn)) - xorigin;
-->     ny = (1:length(yn)) - yorigin;
-->
-->     subplot(2,1,1);
-->     plot2d3(nx, xn);
-->     title("Original signal x(n)");
-->     subplot(2,1,2);
-->     plot2d3(ny, yn);
-->     title("Advanced signal y(n)");
--> endfunction

--> advance ([1,  -2,  3,  6], 3, 1)

 ans = [1x4 double]

   1.  -2.   3.   6.

-->
```



Exercise 3. **function** [yn, yorigin] = **fold** (xn, xorigin) performs folding operation y(n) = x(-n), where

- The discrete-time signal x(n) is presented by vector xn
- xorigin indicates the position of origin in vector x(n).
- yn is vector of the output signal
- yorigin indicates the position of origin in vector y(n).
- x(n) and y(n) are graphically displayed in the same figure.

```
--> exec ('ex3.sci')

--> function [yn, yorigin] = fold(xn, xorigin)
-->     yn = xn($:-1:1);
-->     yorigin = length(xn) - xorigin + 1;
-->     nx = (1:length(xn)) - xorigin;
-->     ny = (1:length(yn)) - yorigin;
-->     subplot(2,1,1);
-->     plot2d3(nx, xn);
-->     title("Original signal x(n)");
-->     subplot(2,1,2);
-->     plot2d3(ny, yn);
-->     title("Folded signal y(n)");
--> endfunction

--> fold ([1,  -2,  3,  6], 3)

 ans = [1x4 double]

   6.   3.  -2.   1.

-->
```



Exercise 4. **function** [yn, yorigin] = **add** (x1n, x1origin, x2n, x2origin) performs addition operation y(n) = x_1(n) + x_2(n), where

- The discrete-time signal x_1(n) and x_2(n) are presented by vectors x1n and x2n, respectively.
- x1origin and x2origin indicate the origin's position of signal x_1(n) and x_2(n), respectively.

- yn is vector of the output signal
- yorigin indicates the origin's position of signal y(n).
- $x_1(n)$, $x_2(n)$ and y(n) are graphically displayed in the same figure.

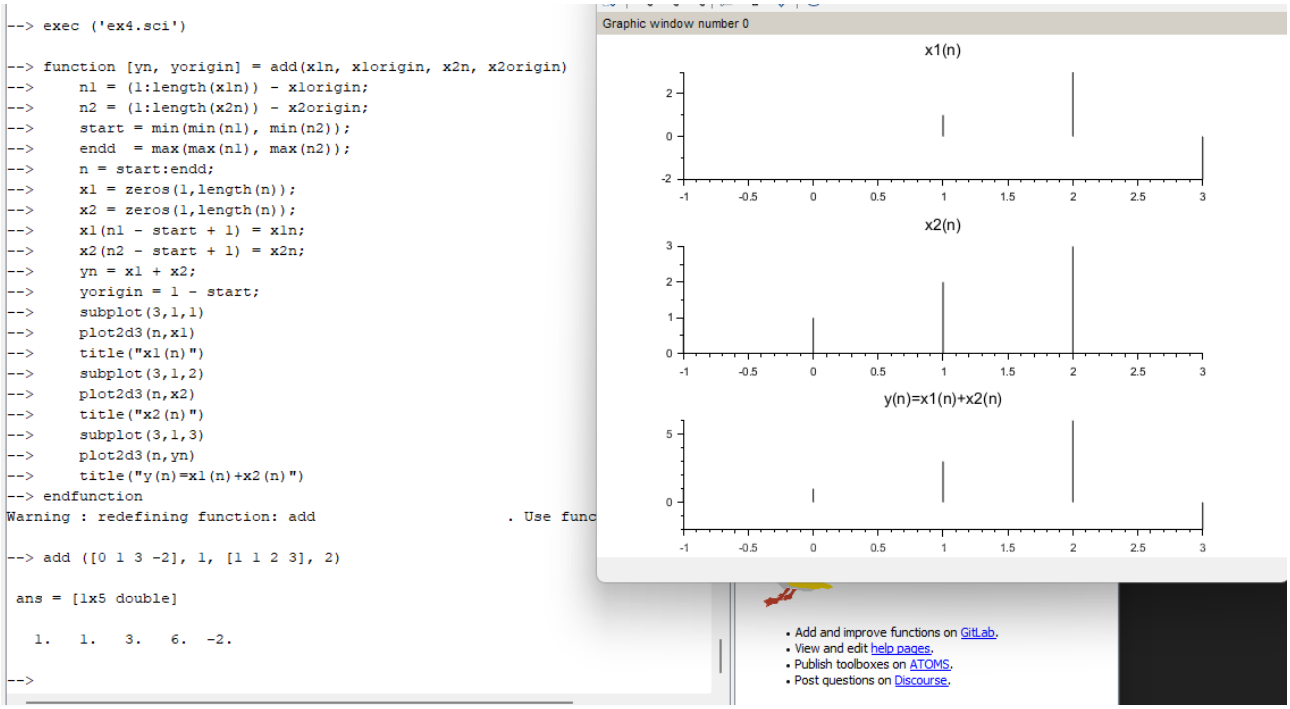
For example: To determine **$y(n) = x_1(n) + x_2(n)$**, where **$x_1(n) = \{0\uparrow, 1, 3, -2\}$** and **$x_2(n)=\{1, 1\uparrow, 2, 3\}$.** You can use your own function by calling

[yn, yorigin] = add ([0, 1, 3, -2], 1, [1, 1, 2, 3], 2**).**

Then, the output will be

- yn = [1, 1, 3, 6, -2]
- yorigin = 1

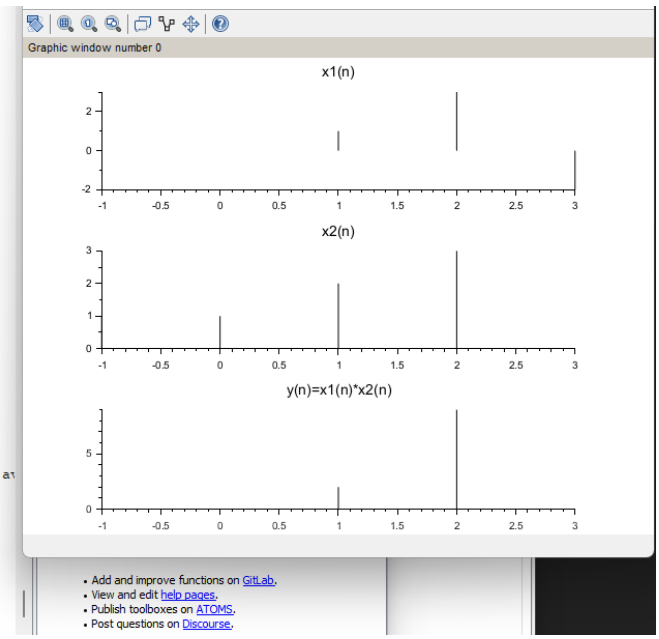
Additionally, you have to graphically display $x_1(n)$, $x_2(n)$ and y(n) in the same figure.



Exercise 5. **function** [yn, yorigin] = **multi** (x1n, x1origin, x2n, x2origin) performs multiplication operation $y(n) = x_1(n).x_2(n)$, where

- The discrete-time signal $x_1(n)$ and $x_2(n)$ are presented by vectors x1n and x2n, respectively.
- x1origin and x2origin indicate the origin's position of signal $x_1(n)$ and $x_2(n)$, respectively.
- yn is vector of the output signal
- yorigin indicates the origin's position of signal y(n).
- $x_1(n)$, $x_2(n)$ and y(n) are graphically displayed in the same figure.

```
--> exec ('ex5.sci')

--> function [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)
-->     n1 = (1:length(x1n)) - x1origin;
-->     n2 = (1:length(x2n)) - x2origin;
-->     start = min(min(n1), min(n2));
-->     endd  = max(max(n1), max(n2));
-->     n = start:endd;
-->     x1 = zeros(1,length(n));
-->     x2 = zeros(1,length(n));
-->     x1(n1 - start + 1) = x1n;
-->     x2(n2 - start + 1) = x2n;
-->     yn = x1 .* x2;
-->     yorigin = 1 - start;
-->     subplot(3,1,1)
-->     plot2d3(n,x1)
-->     title("x1(n)")
-->     subplot(3,1,2)
-->     plot2d3(n,x2)
-->     title("x2(n)")
-->     subplot(3,1,3)
-->     plot2d3(n,yn)
-->     title("y(n)=x1(n)*x2(n)")
--> endfunction
Warning : redefining function: multi                    . Use funcprot(0) to av

--> multi ([0 1 3 -2], 1, [1 1 2 3], 2)

 ans = [1x5 double]

   0.   0.   2.   9.   0.

-->
```

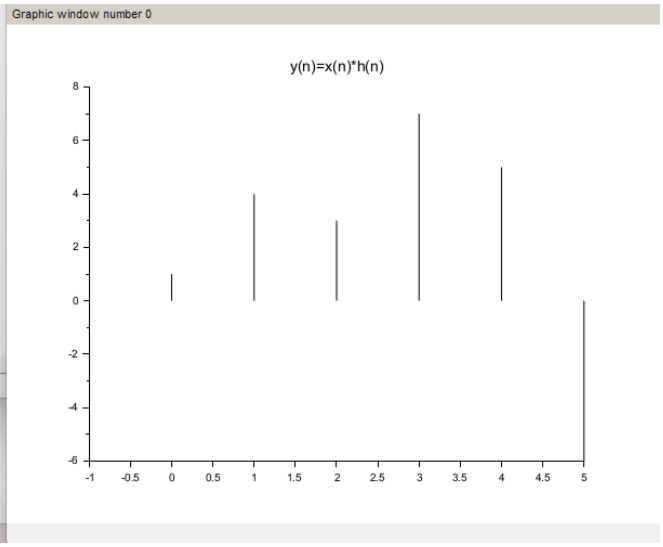


Exercise 6. **function** [yn, yorigin] = **convolution** (xn, xorigin, hn, horigin) performs convolution y(n) = x(n)*h(n), where

- x(n) is the input signal and h(n) is system characteristic's function.
- xorigin and horigin indicates the origin's position of x(n) and h(n), respectively.
- yn is vector of the output signal
- yorigin indicates the origin's position of signal y(n).

```
--> exec ('ex6.sci')

--> function [yn, yorigin] = convolution(xn, xorigin, hn, horigin)
-->     yn = convol(xn, hn);
-->     yorigin = xorigin + horigin - 1;
-->     ny = (1:length(yn)) - yorigin;
-->     plot2d3(ny, yn)
-->     title("y(n)=x(n)*h(n)")
--> endfunction

--> convolution ([0 1 3 -2], 1, [1 1 2 3], 2)

 ans = [1x7 double]

   0.   1.   4.   3.   7.   5.  -6.

-->
```

---------- END ----------



Additional Exercise. Complete exercises 2.1, 2.2, 2.4 and 2.6 in the reference document (both theoretical and practical parts on Scilab) in the attachment.

2.1.

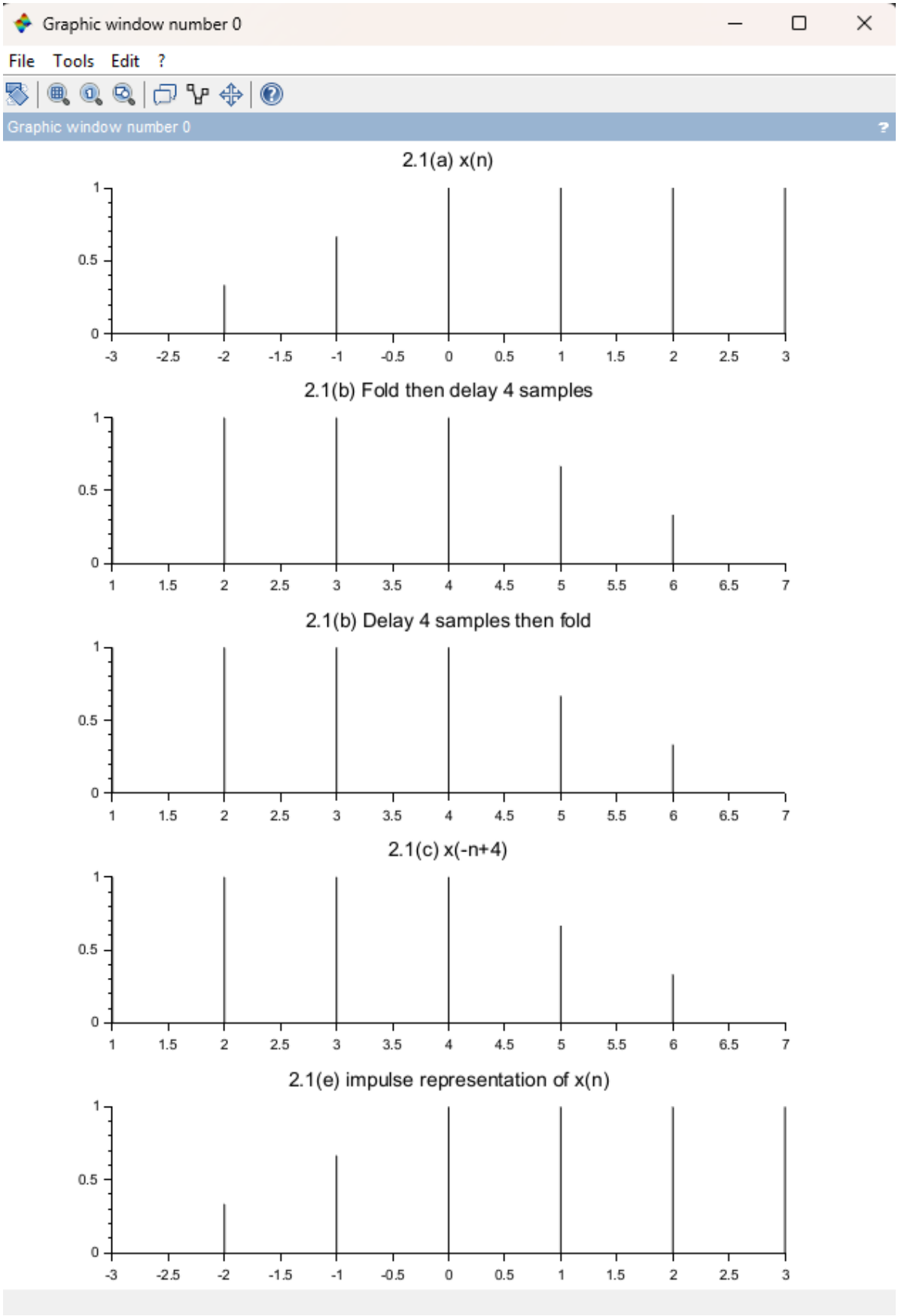
a)

n	-3	-2	-1	0	1	2	3
x(n)	0	1/3	2/3	1	1	1	1

b) Fold rồi delay 4: $x(-n-4)$

Delay 4 rồi fold : $x(-(n-4)) = x(-n+4)$

Vậy ta có 2 phép biến đổi cho ra kết quả khác nhau



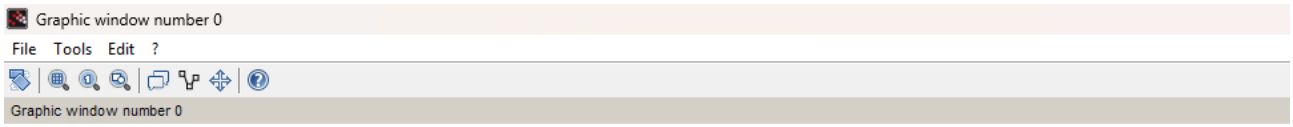
a) Quy luật :

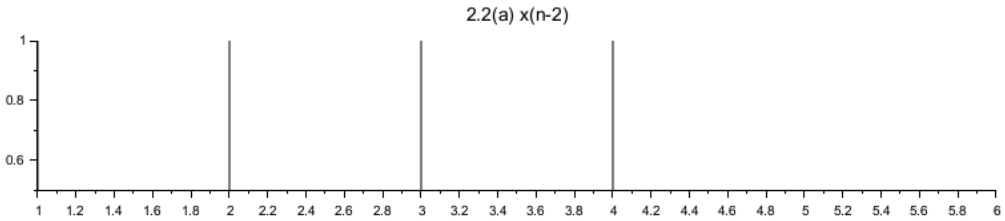
Để tạo $x(-n+k)$ từ $x(n)$ ta delay tín hiệu k mẫu $x(n-k)$ sau đó fold tín hiệu $x\big(-(n-k)\big) = x(-n+k)$

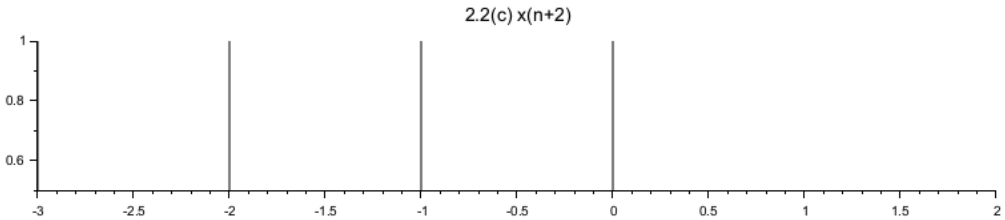
b) $\delta(n)$: $x(n) = \frac{1}{3}\delta(n+2) + \frac{2}{3}\delta(n+1) + \delta(n) + \delta(n-1) + \delta(n-2) + \delta(n-3)$

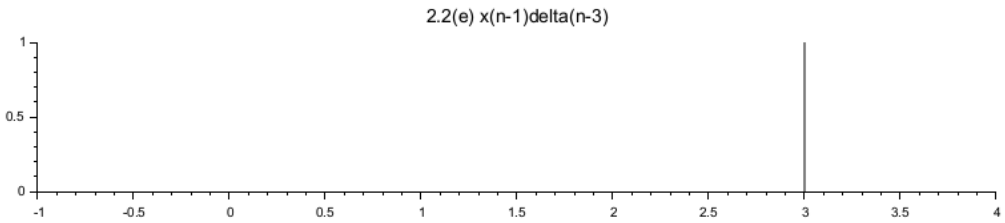
$u(n)$: $x(n) = (1+\frac{n}{3})[u(n+3) - u(n)] + u(n) - u(n-4)$

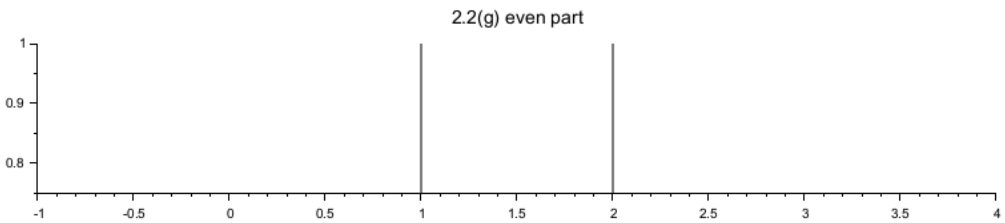
2.2.

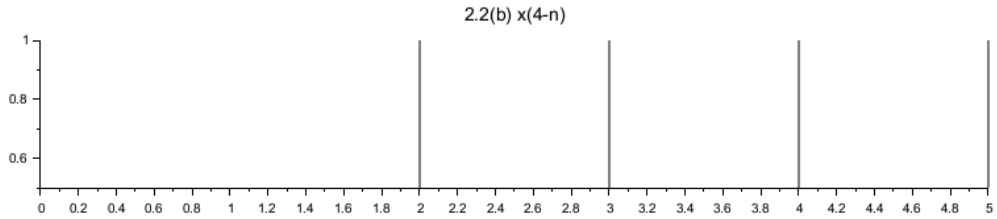


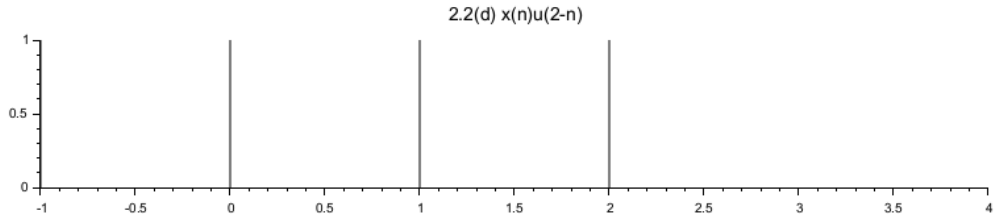


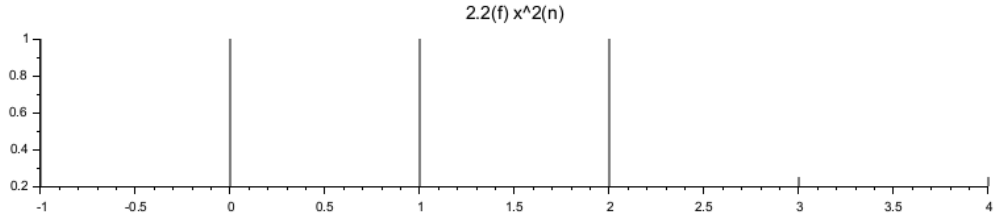


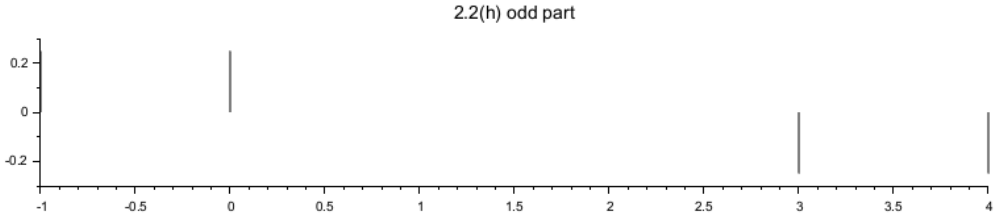












2.4.

Ta cộng hai biểu thức:

$$x_e(n) + x_o(n) = \frac{x(n) + x(-n)}{2} + \frac{x(n) - x(-n)}{2} = \frac{2x(n)}{2} = x(n)$$

Suy ra:

$$x(n) = x_e(n) + x_o(n)$$

Để chứng minh tính duy nhất ta giả sử tồn tại hai cách phân tích:

$$x(n) = x_{e1}(n) + x_{o1}(n)$$
$$x(n) = x_{e2}(n) + x_{o2}(n)$$

Lấy hiệu:

$$x_{e1}(n) - x_{e2}(n) = x_{o2}(n) - x_{o1}(n)$$

Vế trái là even, vế phải là odd

Mà một tín hiệu vừa even vừa odd chỉ có thể là: *0*

Suy ra một tín hiệu chỉ có thể phân tích thành một tín hiệu chẵn và một tín hiệu lẻ.

Vậy phép phân tích là duy nhất.

2.6. $y(n) = T[x(n)] = x(n^2)$

a) Chứng minh time-invariant thỏa: $T[x(n-k)] = y(n-k)$

Ta có: $T[x(n-k)] = x(n^2 - k)$

nhưng: $y(n-k) = x((n-k)^2)$

vì: $x(n^2 - k) \neq x((n-k)^2)$

Vậy hệ time-variant.

b) (1) $x(n) = 1\ khi\ n = 0, 1, 2, 3$

(2) $y(n) = x(n^2)$

n	-2	-1	0	1	2	3	4
n^2	4	1	0	1	4	9	16
$y(n)$	0	1	1	1	0	0	0

(3) $y_2{}'(n) = y(n-2) = x((n-2)^2)$

n	-2	-1	0	1	2	3	4
$(n-2)^2$	16	9	4	1	0	1	4
$y(n-2)$	0	0	0	1	1	1	0

(4) $x_2(n) = x(n-2)$ $x_1(n) = 1$ *khi* $2 \leq n \leq 5$

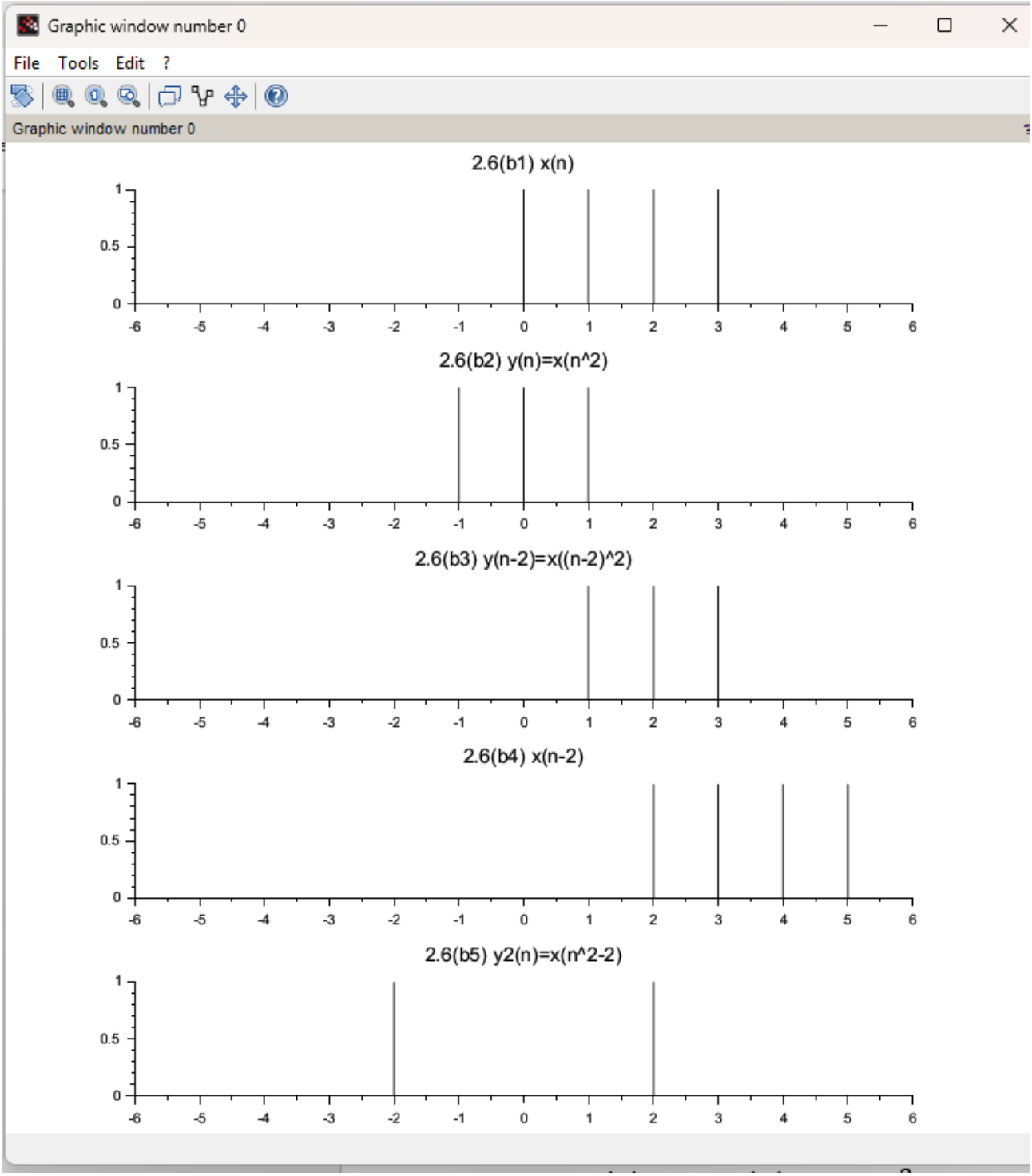
(5) $y_2(n) = x(n^2 - 2)$

vì: $x_1(k) = 1$ khi $2 \leq k \leq 5$

nên: $y_2(n) = 1$ khi $2 \leq n^2 \leq 5 \rightarrow \sqrt{2} \leq |n| \leq \sqrt{5}$

n	-2	-1	0	1	2	3
$y_2(n)$	1	0	0	0	1	0

(6) So sánh $y_2(n)$ *và* $y(n-2)$ *vì* $x(n^2 - 2) \neq x((n-2)^2)$ vậy hệ time-variant.



c) Cho hệ: $y(n) = x(n) - x(n-1)$

$x_1(n) = x(n-k)$

$y_1(n) = T[x(n-k)] = x(n-k) - x(n-k-1)$

$y(n-k) = x(n-k) - x(n-k-1)$

So sánh: $y_1(n) = y(n-k) = x(n-k) - x(n-k-1) \Rightarrow$ hệ time-invariant

d) Cho hệ: $y(n) = n\,x(n)$

$x_1(n) = x(n-k)$

$y_1(n) = T[x(n-k)] = n\,x(n-k)$

$y(n-k) = (n-k)x(n-k)$

So sánh: $n\,x(n-k) \neq (n-k)x(n-k)$ ⇒ hệ không time-invariant

Tài liệu tham khảo:

Github

---------- END ----------